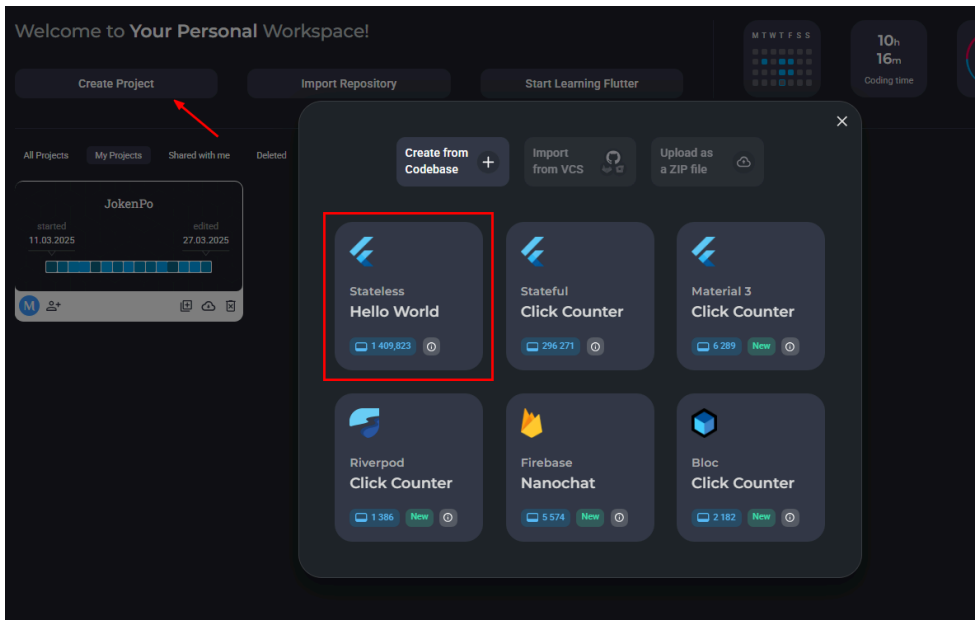


Criando uma Pokedex com Firebase Auth e API Parte 1

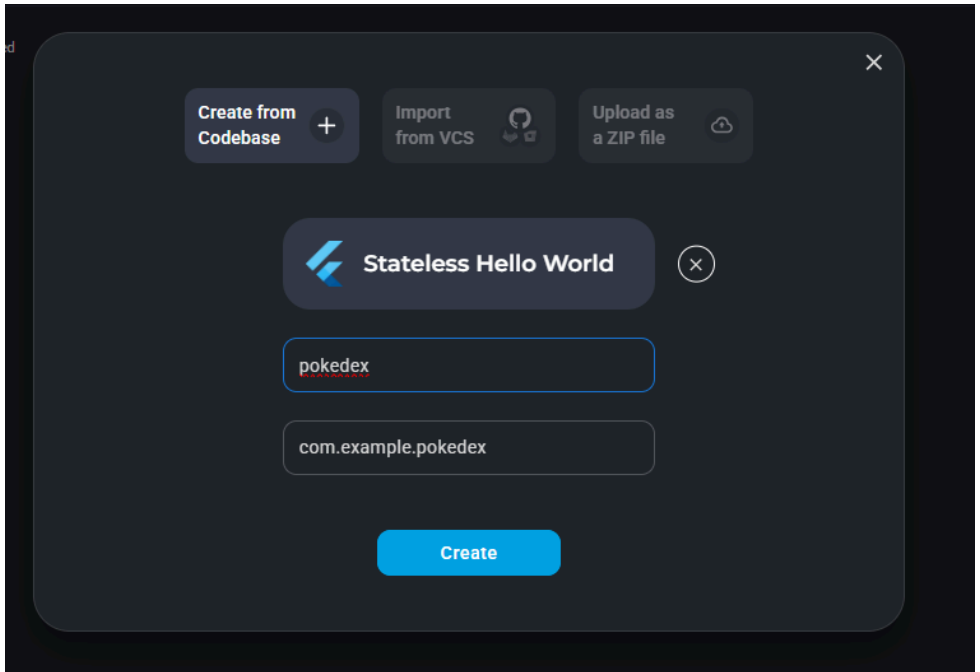
Iniciando o projeto.....	2
Criando o Login.....	7
Adicionando Imagens.....	11
Instalando Pacotes com Pub.dev.....	13
Importando Fonts.....	18
Criando a Tela de Login.....	19
Criando a Tela de Cadastro.....	28
Configurando o Firebase.....	33

Iniciando o projeto

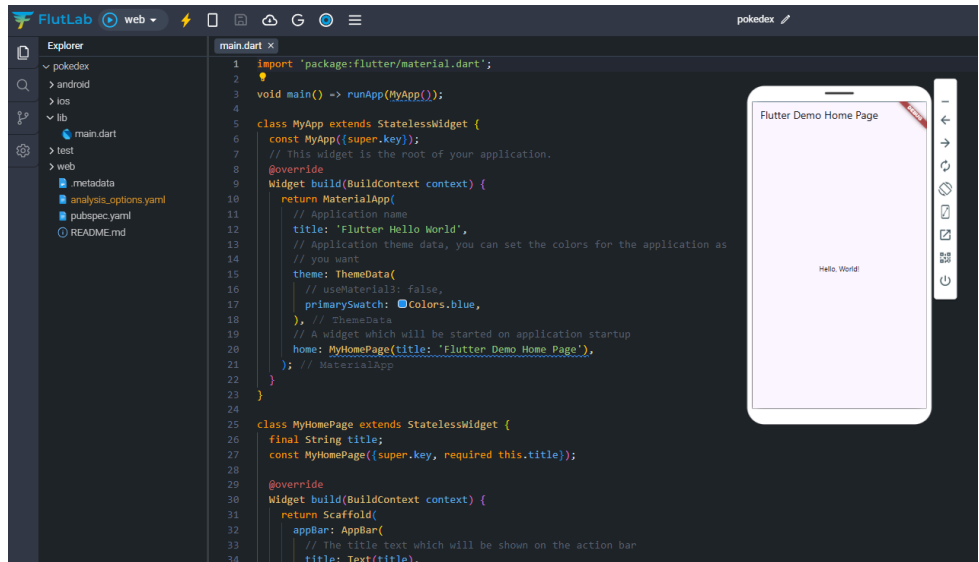
1 - Vamos iniciar um novo projeto no <https://flutlab.io/workspace>, após clicar em Create Project selecione a opção Stateless Hello World:



2 - Coloque o nome de pokedex e depois create:



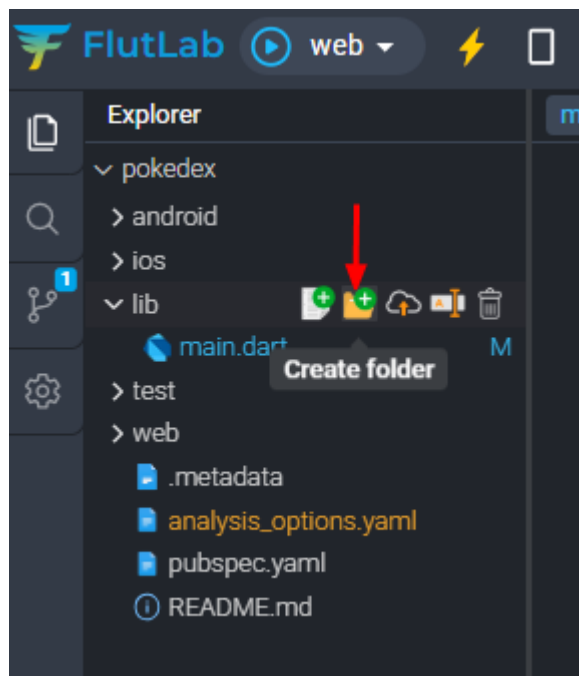
3 - Após abrir seu projeto ele terá essa aparência



4 - Vamos retirar o código referente ao template para iniciar o app do zero, veja que nada aparece no simulador pois de fato em nosso código não estamos passando nenhuma tela para ser exibida.

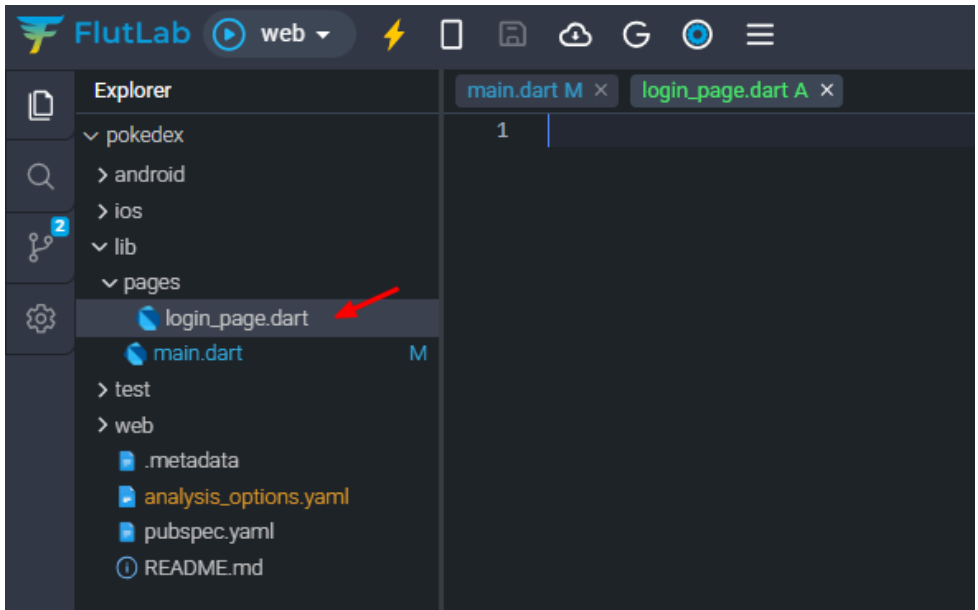
```
main.dart M x
1  import 'package:flutter/material.dart';
2
3  void main() => runApp(MyApp());
4
5  class MyApp extends StatelessWidget {
6      const MyApp({super.key});
7
8      @override
9      Widget build(BuildContext context) {
10         return MaterialApp(
11             debugShowCheckedModeBanner: false,
12         );
13     }
14 }
15
```

5 - Visando boas práticas, vamos criar uma nova pasta dentro de lib para organizar nossas telas. Chame a pasta **pages**:



Criando o Login

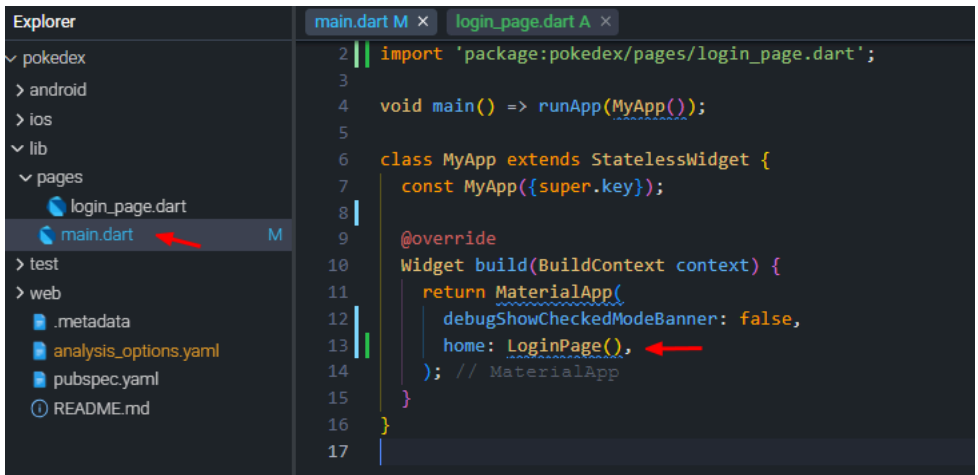
6 - Dentro da pasta pages crie um novo arquivo chamado login_page.dart:



7 - Vamos criar a estrutura básica de uma tela em Flutter, chamaremos a classe dessa tela de LoginPage:

```
main.dart M x login_page.dart A x
1  import 'package:flutter/material.dart';
2
3  class LoginPage extends StatefulWidget {
4      const LoginPage({super.key});
5
6      @override
7      State<LoginPage> createState() => _LoginPageState();
8  }
9
10 class _LoginPageState extends State<LoginPage> {
11     @override
12     Widget build(BuildContext context) {
13         return Scaffold();
14     }
15 }
16
```


8 - Para que a tela de login seja exibida quando o app iniciar vamos voltar ao arquivo main.dart e dentro do MaterialApp vamos adicionar a propriedade home e chamar a classe LoginPage() que criamos anteriormente:

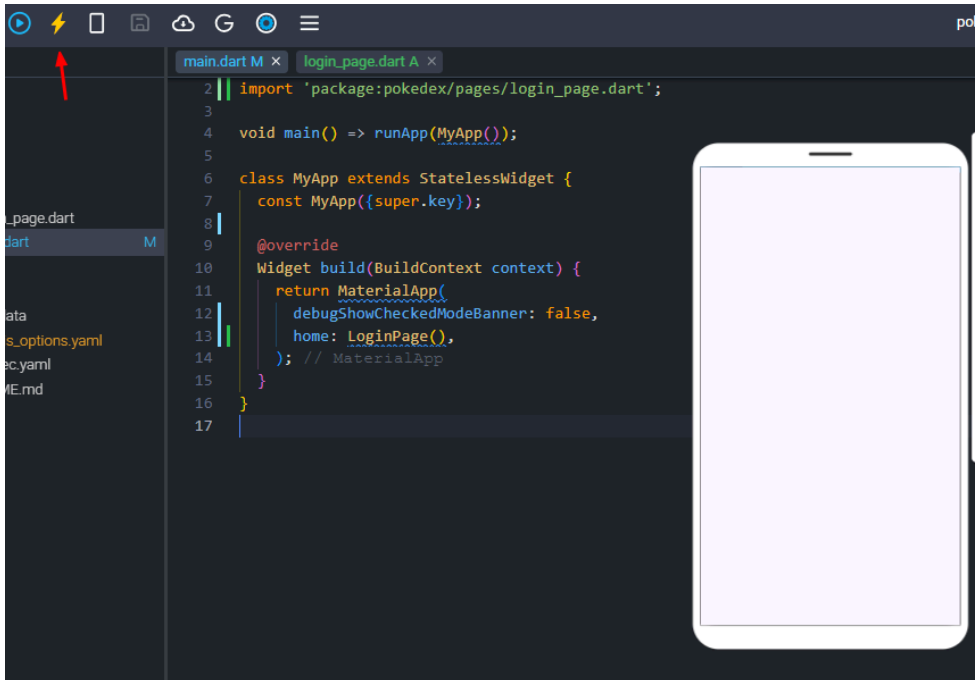


The screenshot shows an IDE with two tabs: 'main.dart M' and 'login_page.dart A'. The 'main.dart' tab is active, showing the following code:

```
2 || import 'package:pokedex/pages/login_page.dart';
3
4 void main() => runApp(MyApp());
5
6 class MyApp extends StatelessWidget {
7   const MyApp({super.key});
8
9   @override
10  Widget build(BuildContext context) {
11    return MaterialApp(
12      debugShowCheckedModeBanner: false,
13      home: LoginPage(),
14    ); // MaterialApp
15  }
16 }
17
```

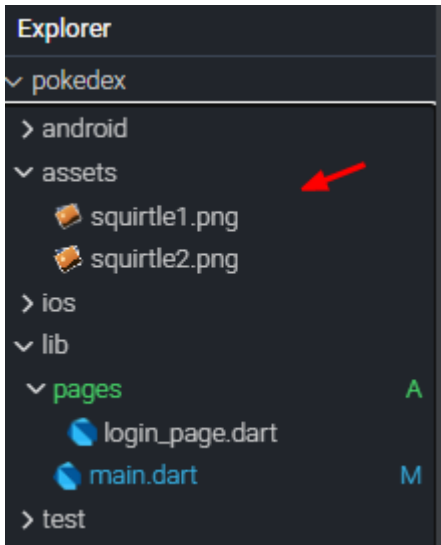
In the left sidebar, the 'Explorer' panel shows the project structure. The 'pages' folder is expanded, and 'main.dart' is selected, indicated by a red arrow. The 'LoginPage()' widget is also highlighted in the code on line 13, also indicated by a red arrow.

9 - Faça o Hot Reload (ou build), veja que agora a aplicação aparece a tela do LoginPage que neste momento é apenas uma tela em branco:

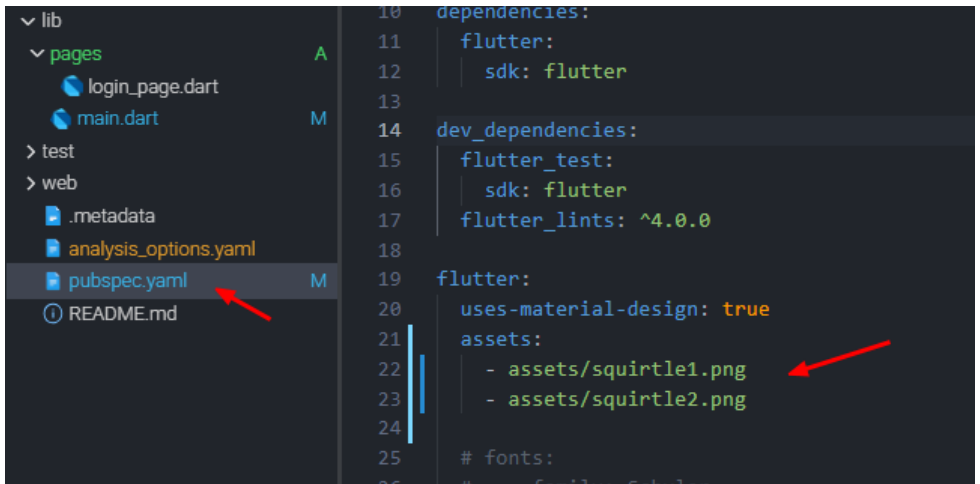


Adicionando Imagens

10 - Crie uma nova pasta chamada assets na raiz do projeto, importe as duas imagens abaixo (AVA) como abaixo:

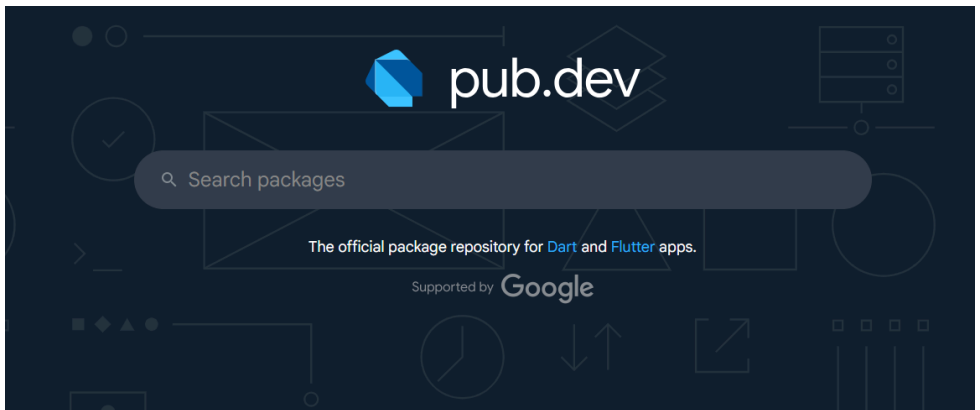


11 - Abra o arquivo pubspec.yaml e vamos colocar as imagens como assets:

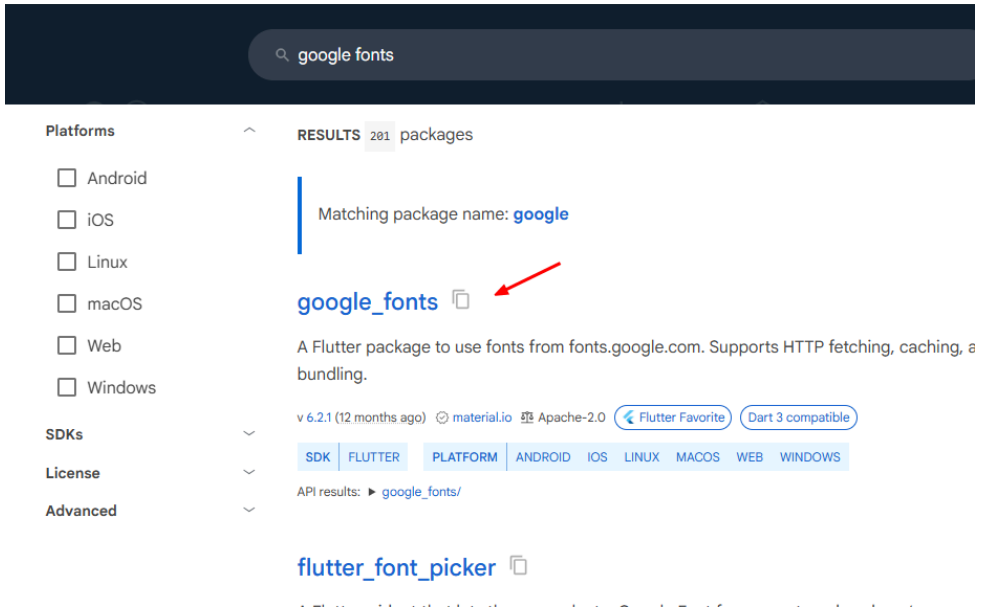


Instalando Pacotes com Pub.dev

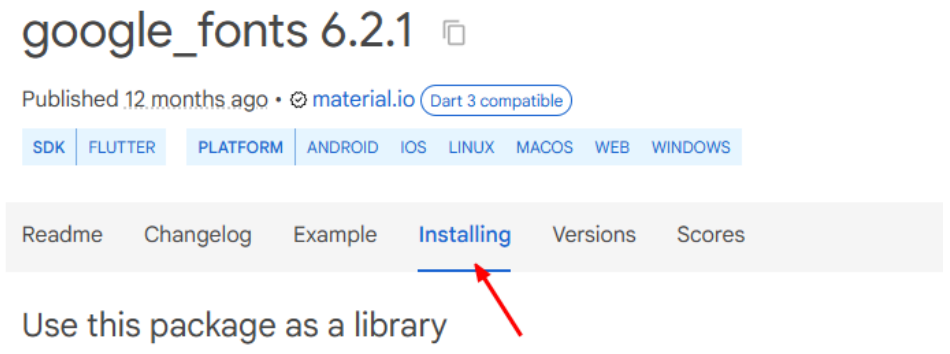
12 - Vamos aproveitar também e trabalhar com fontes customizadas. Sempre que precisar instalar algum package novo visite <https://pub.dev/>, esse é o repositório oficial de packages do Dart e Flutter:



13 - Procure o package google fonts e o abra:



14 - Além de exemplos de uso podemos clicar em Installing e seguir os passos de instalação



Veja que na documentação temos que alizar uma instalação com o seguinte comando:

Use this package as a library

Depend on it

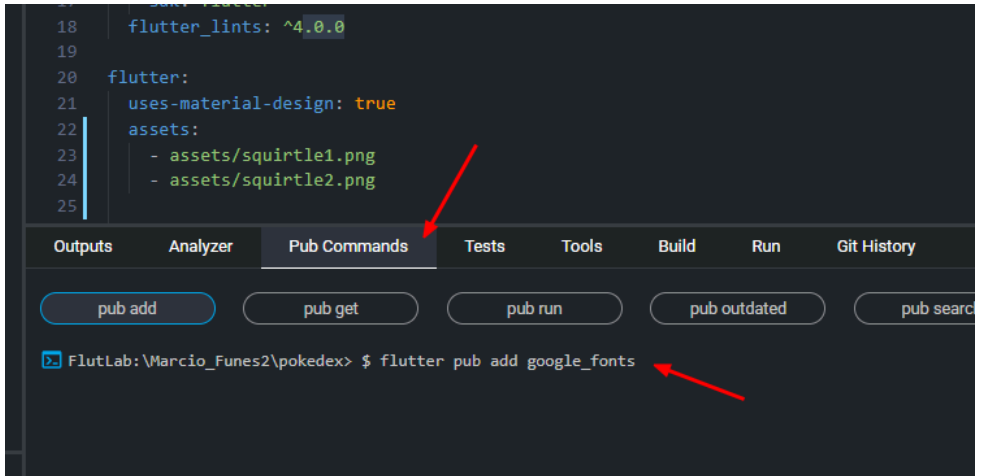
Run this command:

With Flutter:

```
$ flutter pub add google_fonts
```



15 - Na aba Pub Commands execute o comando: **flutter pub add google_fonts**



16 - Veja que o package foi adicionado com sucesso e no pubspec.yaml foi adicionado a versão dessa nova dependência instalada:

```
9
10 dependencies:
11   flutter:
12     sdk: flutter
13   google_fonts: ^6.2.1
14
15 dev_dependencies:
16   flutter_test:
17     sdk: flutter
18   flutter_lints: ^4.0.0
19
20 flutter:
21   uses-material-design: true
22   assets:
23     - assets/squirtle1.png
24     - assets/squirtle2.png
25
```

Outputs Analyzer Pub Commands Tests Tools Build Run Git Hi

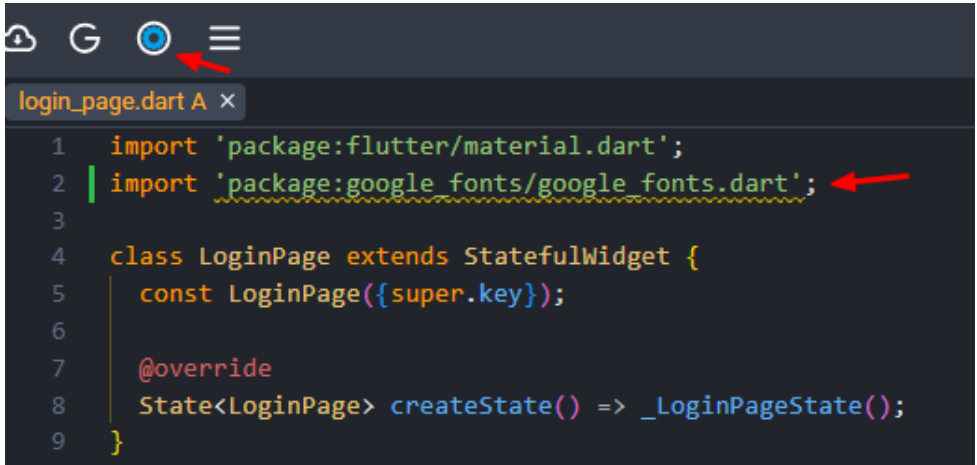
pub add pub get pub run pub outdated

Running "flutter pub add" in FlutLab\Marcio_Funes2\pokedex>

✓ Package added successfully. Run "pub get" command

Importando Fonts

17 - Agora podemos no arquivo login_page.dart importar o google fonts e usar em nosso desenvolvimento. Caso essa linha apresente algum erro, clique em Restart Analyser para correção:



The screenshot shows an IDE window titled 'login_page.dart A x'. The code is as follows:

```
1 import 'package:flutter/material.dart';
2 import 'package:google_fonts/google_fonts.dart';
3
4 class LoginPage extends StatefulWidget {
5   const LoginPage({super.key});
6
7   @override
8   State<LoginPage> createState() => _LoginPageState();
9 }
```

Two red arrows point to the import statements: one to the Flutter import on line 1 and another to the Google Fonts import on line 2. The Google Fonts import line has a yellow wavy underline, indicating a warning or error.

Criando a Tela de Login

18 - Vamos agora customizar nossa tela de login, vamos usar essa tela como referência:

 **Pokedex**

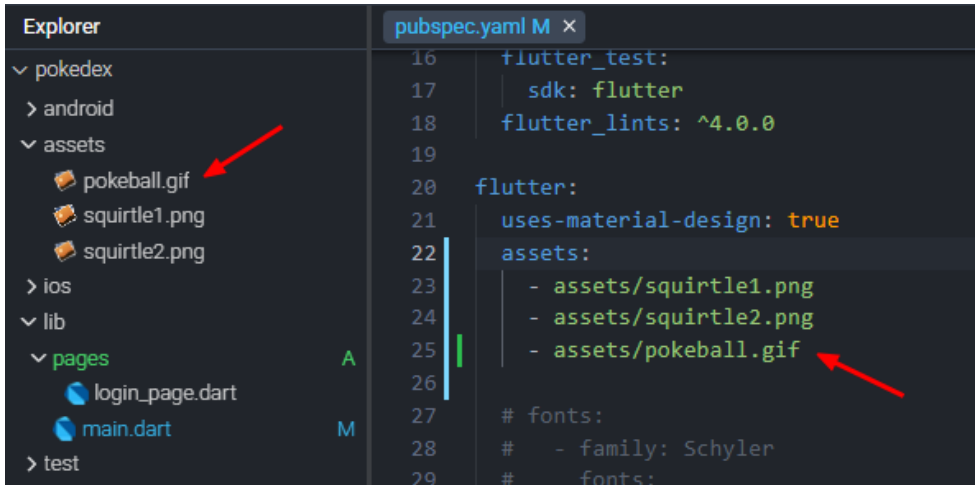


Login

Login

Não tem um cadastro? [Cadastre-se](#)

19 - Adicione um gif aos assets (AVA) e também no pubspec.yaml



20 - Nas linhas 1 ao 20 temos a estrutura inicial do app, as caixas de texto para email e senha que por hora apenas vão printar no console:

```
login_page.dart A x
1  import 'package:flutter/material.dart';
2  import 'package:google_fonts/google_fonts.dart';
3
4  class LoginPage extends StatefulWidget {
5      const LoginPage({super.key});
6
7      @override
8      State<LoginPage> createState() => _LoginPageState();
9  }
10
11  class _LoginPageState extends State<LoginPage> {
12      final TextEditingController _emailController = TextEditingController();
13      final TextEditingController _passwordController = TextEditingController();
14
15      void _login() {
16          String email = _emailController.text;
17          String password = _passwordController.text;
18          print('Email: $email');
19          print('Password: $password');
20      }
21
```

21 - Nas linhas 22 a 45 temos a adição do Scaffold que organiza o appBar (barra no topo) com a imagem da pokebola, o título Pokedex e a cor de fundo vermelha.

```
21
22  @override
23  Widget build(BuildContext context) {
24    return Scaffold(
25      appBar: AppBar(
26        title: Row(
27          children: [
28            Image.asset(
29              'assets/pokeball.gif',
30              height: 40,
31              width: 40,
32            ), // Image.asset
33            const SizedBox(width: 8),
34            Text(
35              'Pokedex',
36              style: GoogleFonts.limelight(
37                fontSize: 24,
38                fontWeight: FontWeight.bold,
39              ),
40            ), // Text
41          ],
42        ), // Row
43        backgroundColor: Colors.red,
44        elevation: 0,
45      ), // AppBar
```

22 - Das linhas 46 até 72 temos a definição da cor do fundo (linha 47), a adição da imagem centralizada, o texto login. Veja que estamos usando o recurso GoogleFonts. o nome da fonte que desejar:

Consulte as fontes disponíveis: <https://fonts.google.com/>

```
44     elevation: 0,
45   ), // AppBar
46   body: Container(
47     color: Color(0xffffffff),
48     child: Padding(
49       padding: const EdgeInsets.all(16.0),
50       child: Column(
51         mainAxisAlignment: MainAxisAlignment.center,
52         children: <Widget>[
53           Center(
54             child: Image.asset(
55               'assets/squirtle1.png',
56               height: 200,
57               width: 200,
58             ), // Image.asset
59           ), // Center
60           const SizedBox(height: 32),
61           Align(
62             alignment: Alignment.centerLeft,
63             child: Text(
64               'Login',
65               style: GoogleFonts.dmSerifText(
66                 fontSize: 28,
67                 fontWeight: FontWeight.bold,
68                 color: Color(0xff000000),
69               ),
70             ), // Text
71           ), // Align
72           const SizedBox(height: 32),
```

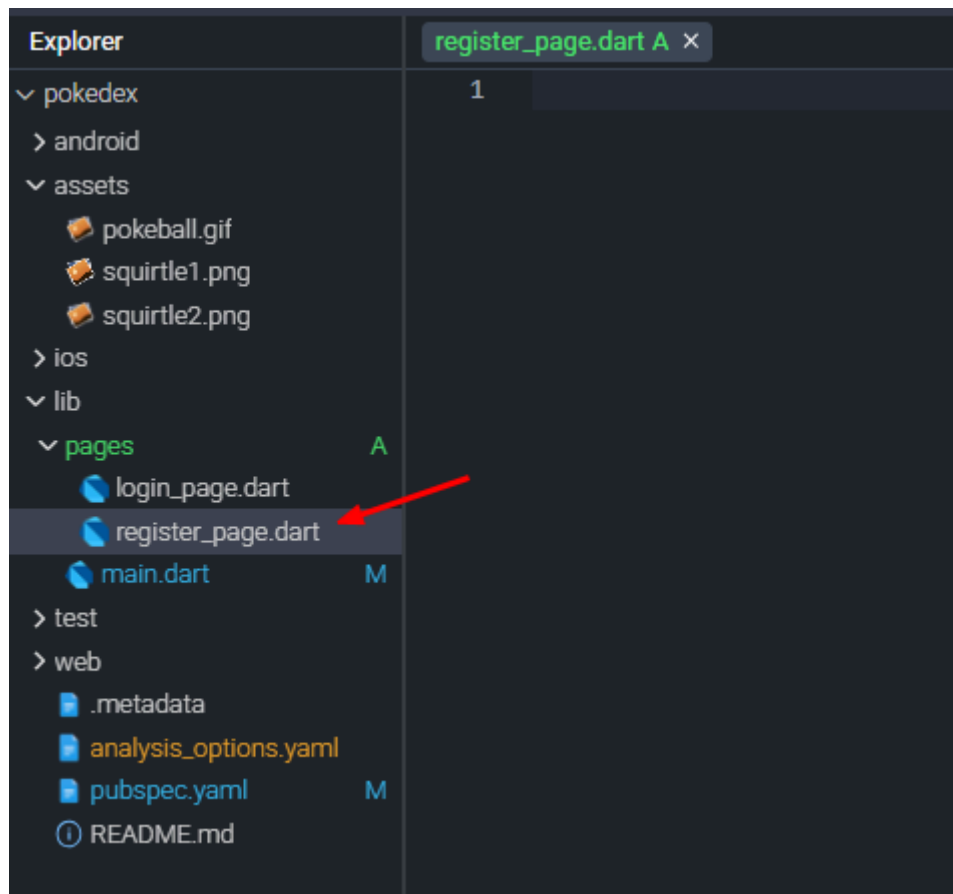
23 - Nas linhas 73 até 102 temos o texto que irá dentro da caixa de texto Email e Password além de colocarmos o botão de cadastro na cor vermelha:

```
70     ), // Text
71   ), // Align
72   const SizedBox(height: 32),
73   TextField(
74     controller: _emailController,
75     decoration: const InputDecoration(
76       labelText: 'Email',
77       labelStyle: TextStyle(color: Colors.grey),
78       border: OutlineInputBorder(),
79     ), // InputDecoration
80     keyboardType: TextInputType.emailAddress,
81   ), // TextField
82   const SizedBox(height: 16),
83   TextField(
84     controller: _passwordController,
85     decoration: const InputDecoration(
86       labelText: 'Password',
87       labelStyle: TextStyle(color: Colors.grey),
88       border: OutlineInputBorder(),
89     ), // InputDecoration
90     obscureText: true,
91   ), // TextField
92   const SizedBox(height: 16),
93   ElevatedButton(
94     onPressed: _login,
95     child: const Text(
96       'Login',
97       style: TextStyle(color: Colors.black),
98     ), // Text
99     style: ElevatedButton.styleFrom(
100       backgroundColor: Colors.red,
101     ),
102   ), // ElevatedButton
```


24 - Na linha 103 a 123 temos o texto de 'Não tem um cadastro?' e um link para levar o usuário a outra tela:

```
100         backgroundColor: Colors.red,
101     ),
102 ), // ElevatedButton
103 const SizedBox(height: 32),
104 Row(
105     mainAxisAlignment: MainAxisAlignment.center,
106     children: [
107         const Text('Não tem um cadastro?'),
108         TextButton(
109             onPressed: () {},
110             child: const Text(
111                 'Cadastre-se',
112                 style: TextStyle(color: Colors.blue),
113             ), // Text
114         ), // TextButton
115     ],
116 ), // Row
117 ], // <Widget>[]
118 ), // Column
119 ), // Padding
120 ), // Container
121 ); // Scaffold
122 }
123 }
124
```

25 - Crie um novo arquivo chamado register_page.dart na pasta pages:



Criando a Tela de Cadastro

26 - Copie todo o código do arquivo login_page.dart para o register_page.dart. Essa será a tela de cadastro:





Cadastrar

Cadastro

Pokémon! Temos que pegar

vamos fazer algumas pequenas alterações:

```
register_page.dart A x
1  import 'package:flutter/material.dart';
2  import 'package:google_fonts/google_fonts.dart';
3
4  class RegisterPage extends StatefulWidget {
5      const RegisterPage({super.key});
6
7      @override
8      State<RegisterPage> createState() => _RegisterPageState();
9  }
10
11  class _RegisterPageState extends State<RegisterPage> {
12      final TextEditingController _emailController = TextEditingController();
13      final TextEditingController _passwordController = TextEditingController();
14
15      void _login() {
16          String email = _emailController.text;
17          String password = _passwordController.text;
18          print('Email: $email');
19          print('Password: $password');
20      }
21  }
```

```
21
22 @override
23 Widget build(BuildContext context) {
24   return Scaffold(
25     appBar: AppBar(
26       backgroundColor: Colors.red,
27       elevation: 0,
28     ), // AppBar
29     body: Container(
30       color: Color(0xffffffff),
31       child: Padding(
32         padding: const EdgeInsets.all(16.0),
33         child: Column(
34           mainAxisAlignment: MainAxisAlignment.center,
35           children: <Widget>[
36             Center(
37               child: Image.asset(
38                 'assets/squirtle2.png',
39                 height: 200,
40                 width: 200,
41               ), // Image.asset
42             ), // Center
43             const SizedBox(height: 32),
44             Align(
45               alignment: Alignment.centerLeft,
46               child: Text(
47                 'Cadastrar',
48                 style: GoogleFonts.dmSerifText(
49                   fontSize: 28,
50                   fontWeight: FontWeight.bold,
51                   color: Color(0xff000000),
52                 ),
53               ), // Text
54             ), // Align
```

```

53     ), // Text
54   ), // Align
55   const SizedBox(height: 32),
56   TextField(
57     controller: _emailController,
58     decoration: const InputDecoration(
59       labelText: 'Digite seu email:',
60       labelStyle: TextStyle(color: Colors.grey),
61       border: OutlineInputBorder(),
62     ), // InputDecoration
63     keyboardType: TextInputType.emailAddress,
64   ), // TextField
65   const SizedBox(height: 16),
66   TextField(
67     controller: _passwordController,
68     decoration: const InputDecoration(
69       labelText: 'Digite seu Password',
70       labelStyle: TextStyle(color: Colors.grey),
71       border: OutlineInputBorder(),
72     ), // InputDecoration
73     obscureText: true,
74   ), // TextField
75   const SizedBox(height: 16),
76   ElevatedButton(
77     onPressed: _login,
78     child: const Text(
79       'Cadastro',
80       style: TextStyle(color: Colors.black),
81     ), // Text
82     style: ElevatedButton.styleFrom(
83       backgroundColor: Colors.red,
84     ),
85   ), // ElevatedButton
86   const SizedBox(height: 32),
87   Row(
88     mainAxisAlignment: MainAxisAlignment.center,
89     children: [
90       const Text('Pokémon! Temos que pegar'),
91       const SizedBox(height: 32),
92     ],

```

```
88         mainAxisAlignment: MainAxisAlignment.center,
89         children: [
90             const Text('Pokémon! Temos que pegar'),
91             const SizedBox(height: 32),
92         ],
93     ), // Row
94 ], // <Widget>[]
95 ), // Column
96 ), // Padding
97 ), // Container
98 ); // Scaffold
99 }
100 }
101
```

Configurando o Firebase

27 - Agora que temos as telas de login e cadastro vamos configurar o back-end da nossa aplicação, acesse o <https://firebase.google.com/?hl=pt-br> e crie uma conta ou use sua conta Google.

Faça seu app  o melhor possível com o Firebase e a IA generativa

Crie e execute experiências modernas com tecnologia de IA que os usuários adoram com o Firebase, uma plataforma projetada para ajudar você durante toda a jornada de desenvolvimento de apps. Com o apoio do Google e a confiança de milhões de empresas no mundo todo.

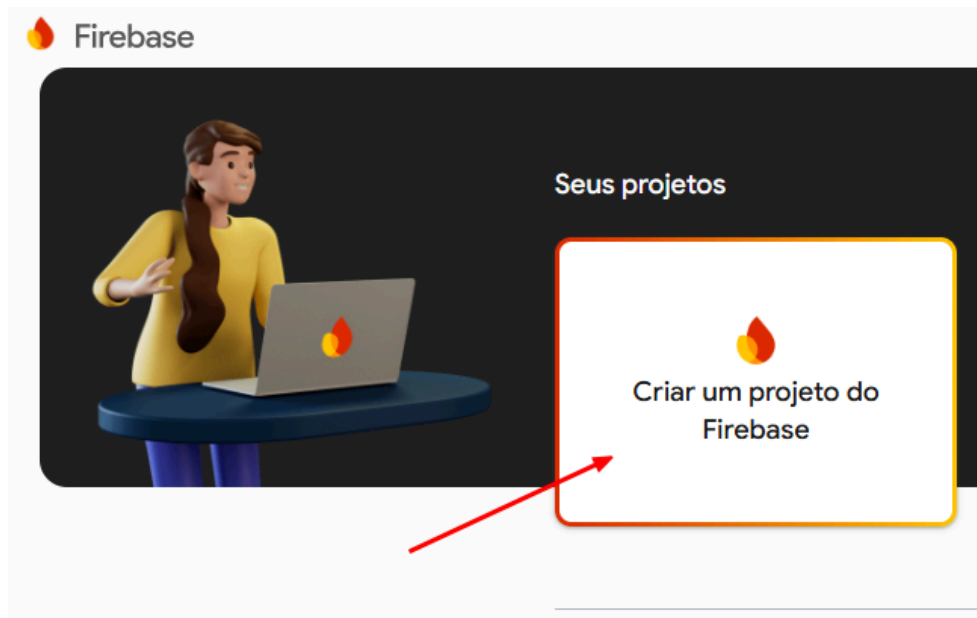
[Começar](#)

[Teste a demonstração →](#)

28 - Após login clique em Go to console:



29 - Clique em Criar um projeto Firebase:



30 - Dê um nome ao seu projeto e clique em continuar:

× Criar um projeto

Vamos começar nomeando o projeto[?]

Nome do projeto

pokedex-app

 pokedex-app-12f28

 unifacef.com.br

Já tem um projeto do Google Cloud?
[Adicionar o Firebase ao projeto do Google Cloud](#)

Continuar

31 - Desabilite o Analytics e clique em Criar projeto:

× Criar um projeto

Google Analytics para seu projeto do Firebase

O Google Analytics é uma solução de análise ilimitada e gratuita. Com ele, é possível segmentar, gerar relatórios e muito mais nos seguintes produtos: Firebase Crashlytics, Cloud Messaging, Mensagens no app, Configuração remota, Teste A/B e Cloud Functions.

O Google Analytics ativa:

× Teste A/B ⓘ

× Gatilhos do Cloud Functions com base ⓘ
em eventos

× Segmentação de usuários em produtos ⓘ
do Firebase

× Geração de relatórios ilimitada gratuita ⓘ

× Registros de navegação estrutural no ⓘ
Crashlytics

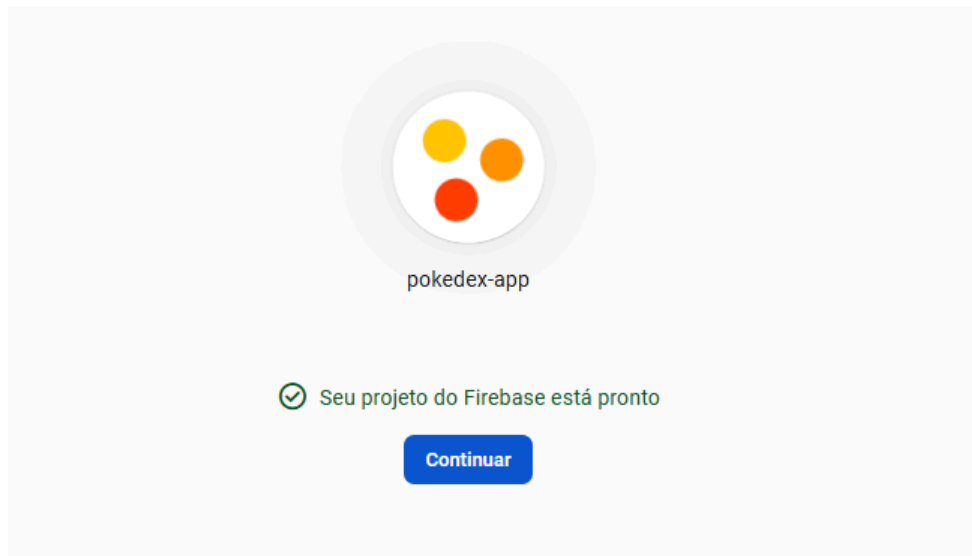


Ativar o Google Analytics neste projeto
Recomendado

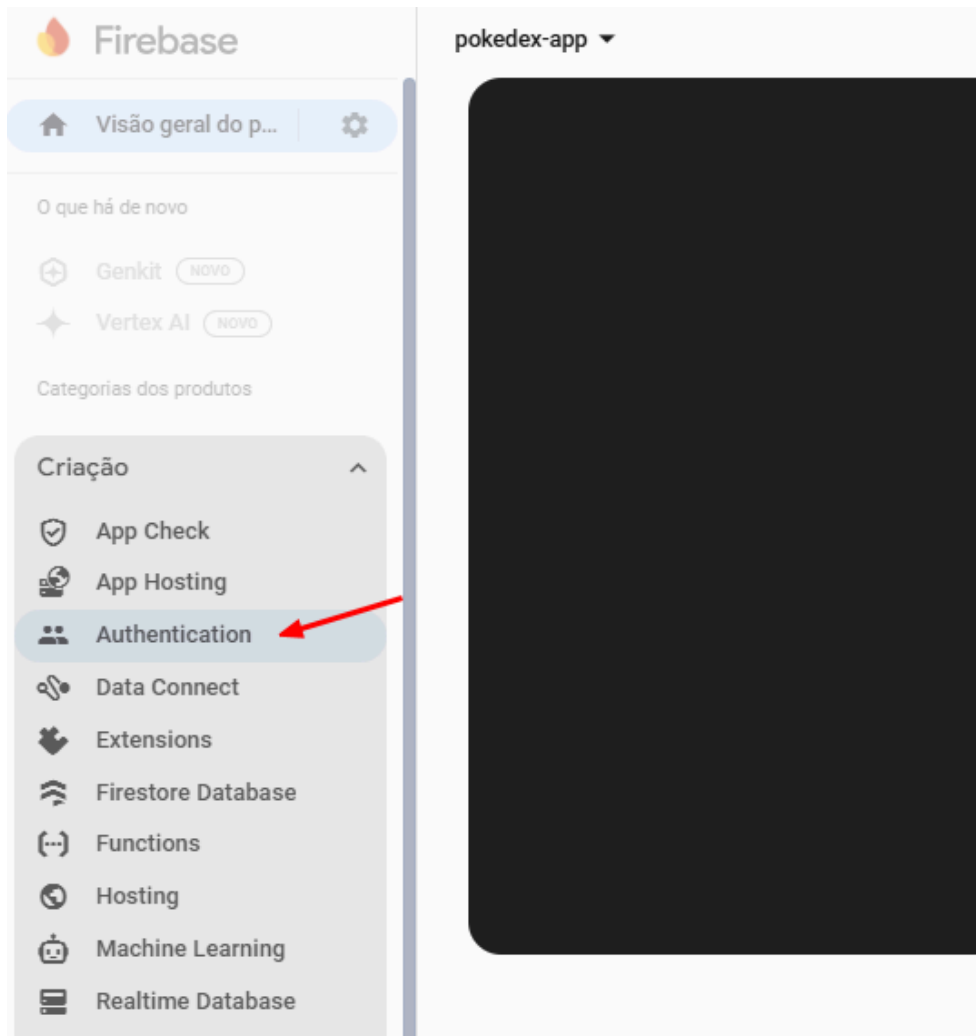
[Anterior](#)

Criar projeto

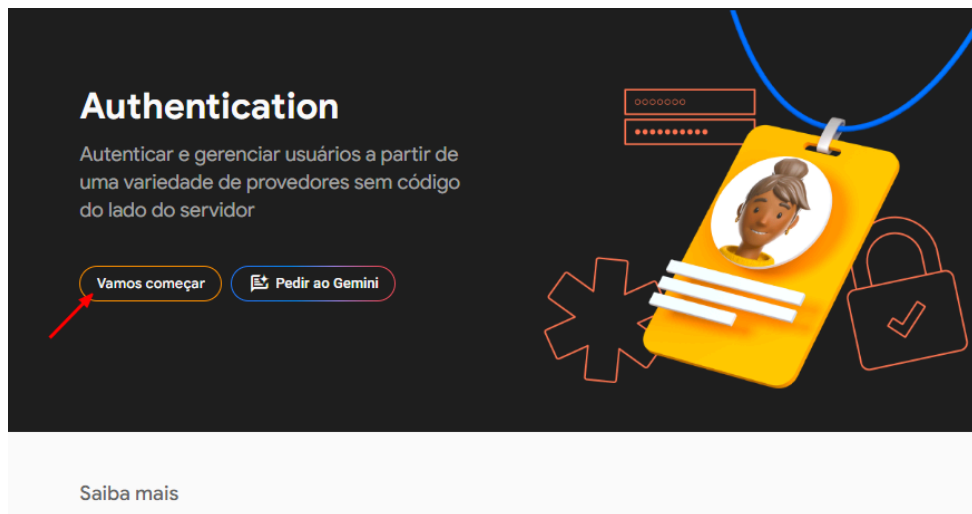
32 - Aguarde a criação do projeto e clique em continuar:



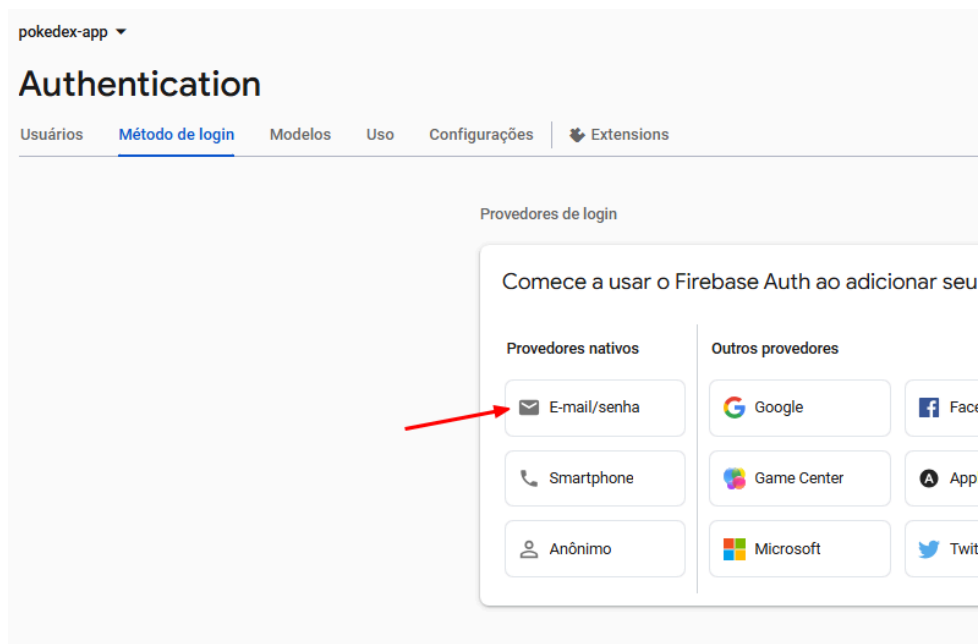
33 - Clique em Authentication:



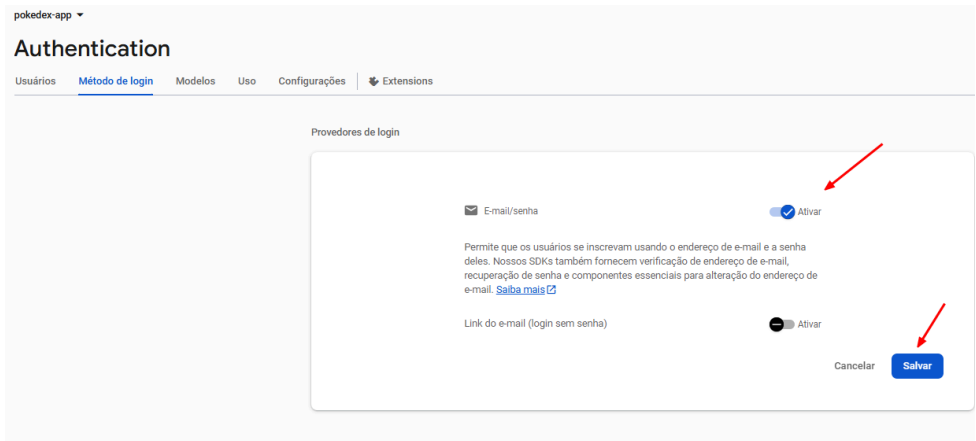
34 - Clique em Vamos começar:



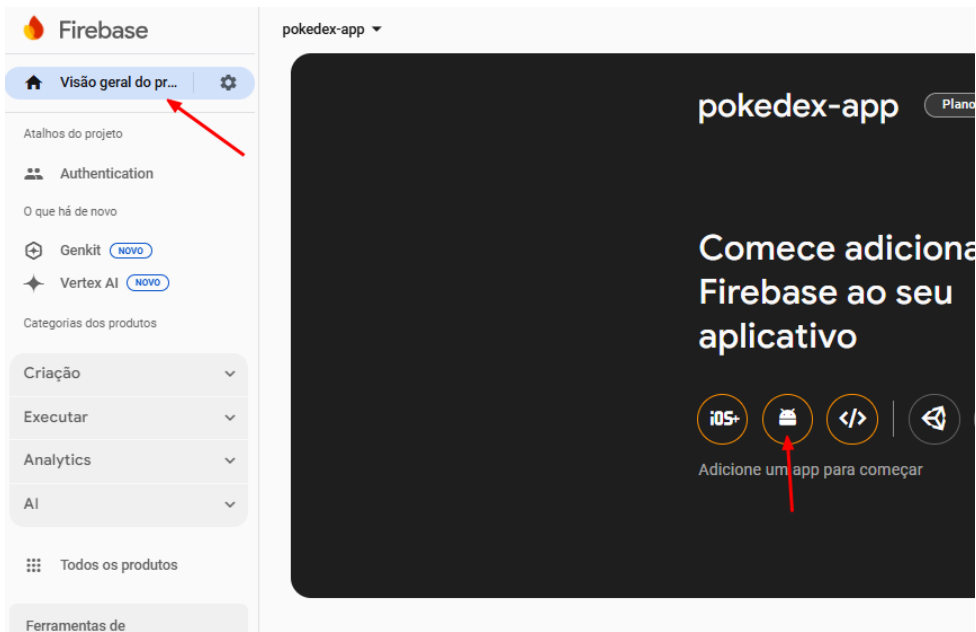
35 - Clique em Email/senha:



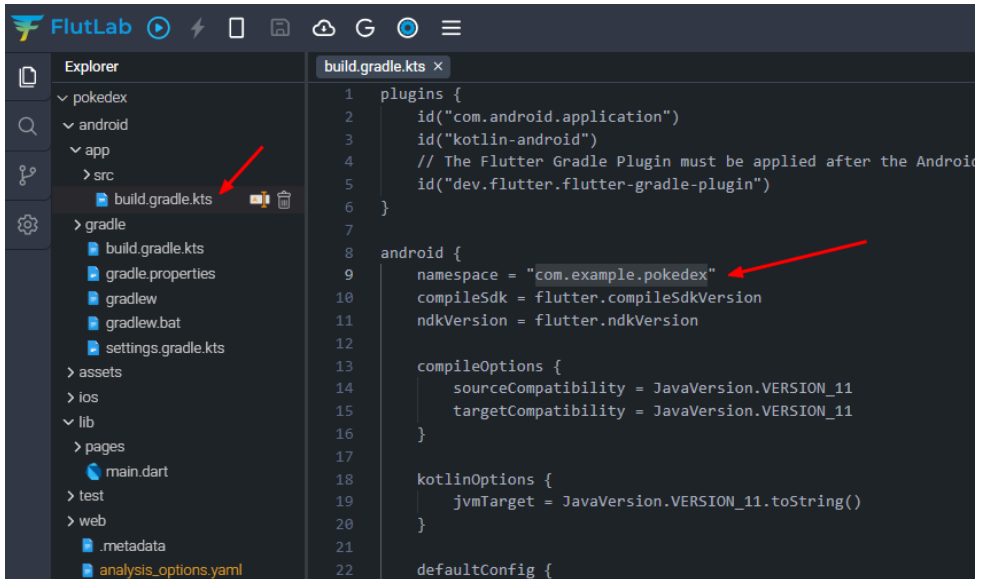
36 - Clique em ativar e depois salvar:



37 - Clique em Visão geral do projeto e depois em Android (ou IOS+ caso use iPhone):



38 - Coloque o nome do pacote do seu aplicativo. Caso tenha esquecido o nome escolhido durante a criação do projeto:



1

②

com.example.pokedex

②

Meu app Android

②

00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00

Registrar app

2

3

4

39 - Dê um apelido ao app e depois Registrar:

✕ Adicionar o Firebase ao seu app Android

1 Registrar app

Nome do pacote do Android (?)

com.example.pokedex

Apelido do app(opcional)

Pokedex App

Certificado de assinatura SHA-1 de depuração (opcional) ?

00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:(

ⓘ **Necessário para os Dynamic Links, o Login do Google e o suporte por telefone no Auth. Edite os certificados SHA-1 nas configurações.**

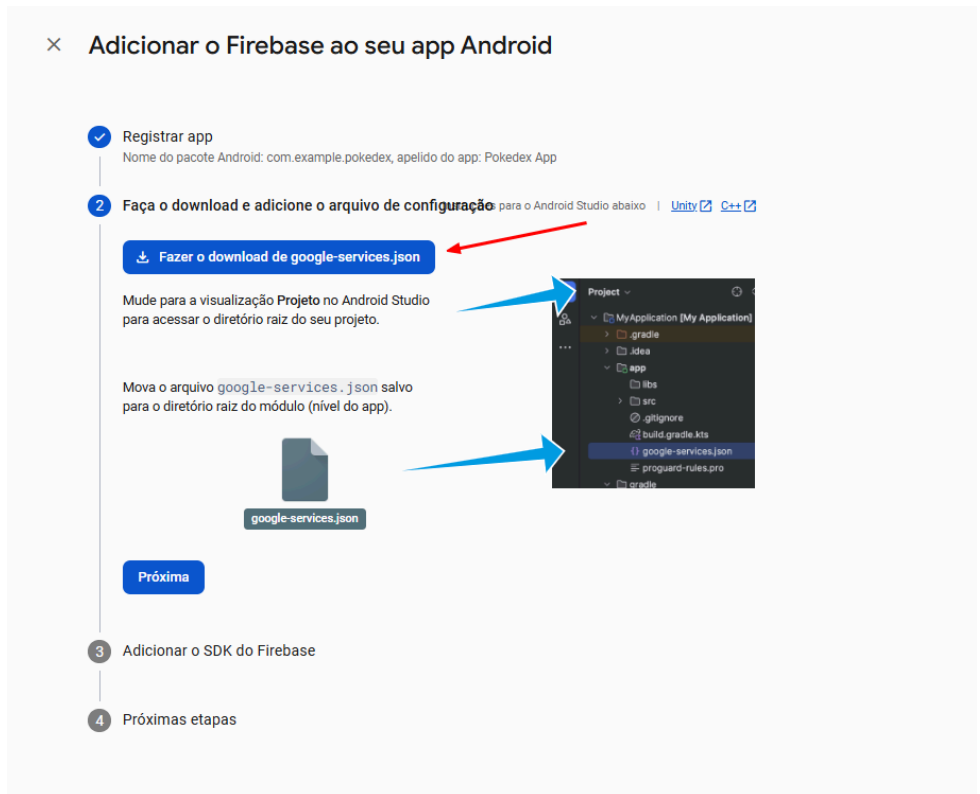
Registrar app

2 Faça o download e adicione o arquivo de configuração

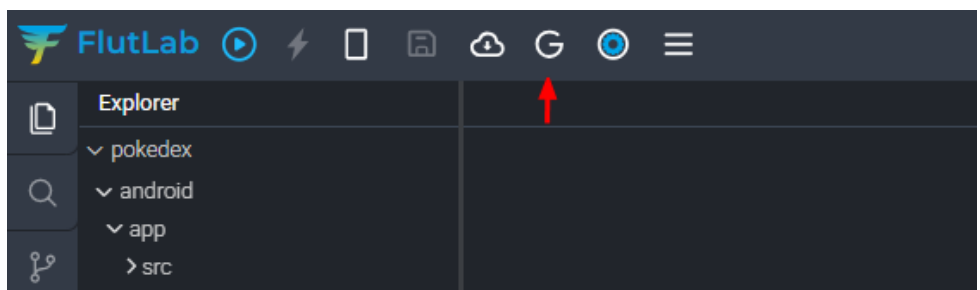
3 Adicionar o SDK do Firebase

4 Próximas etapas

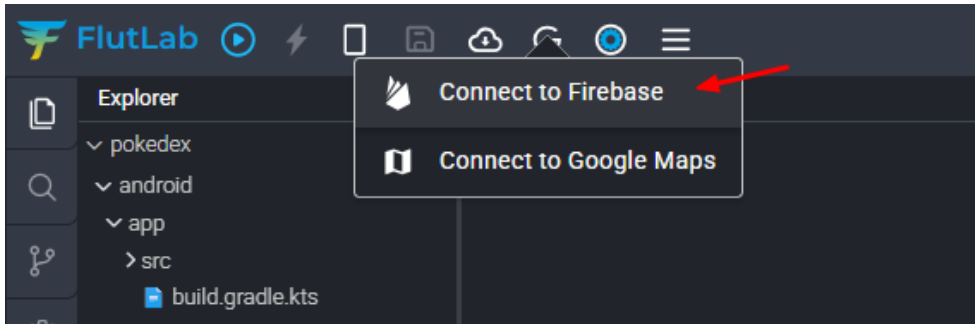
40 - Faça o download do google-service.json, iremos utilizar esse arquivo na sequência, não o perca. Dê próxima até retornar para a tela do projeto:



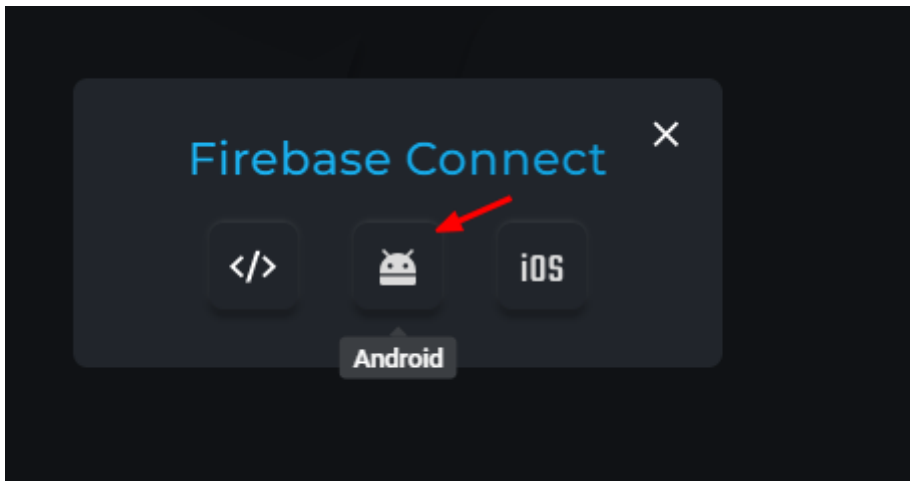
41 - Clique Google Service:



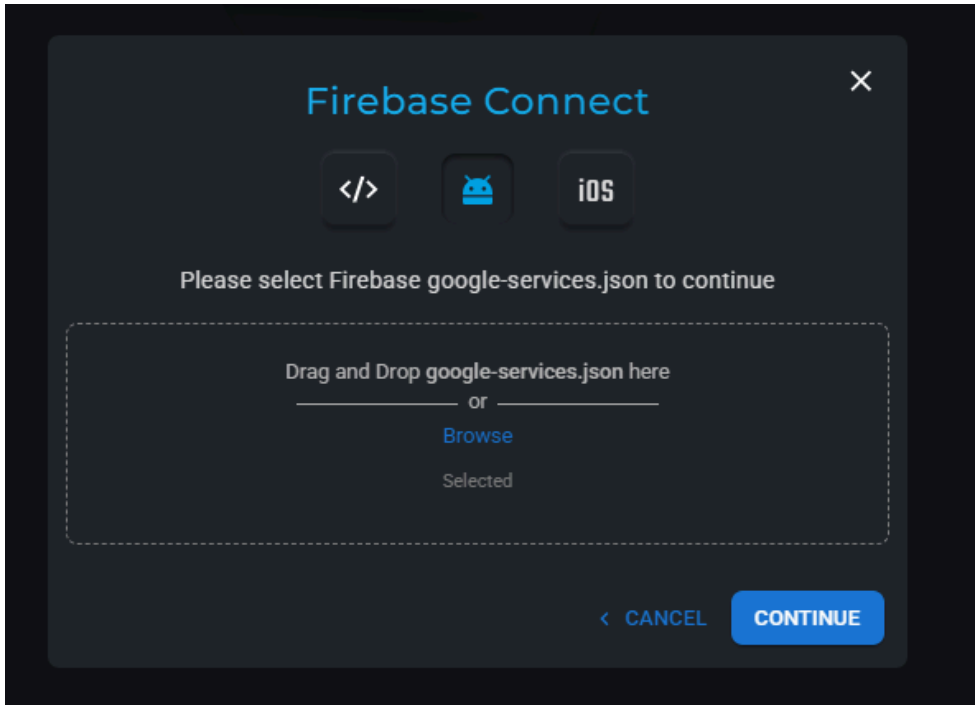
42 - Clique em Connect to Firebase:



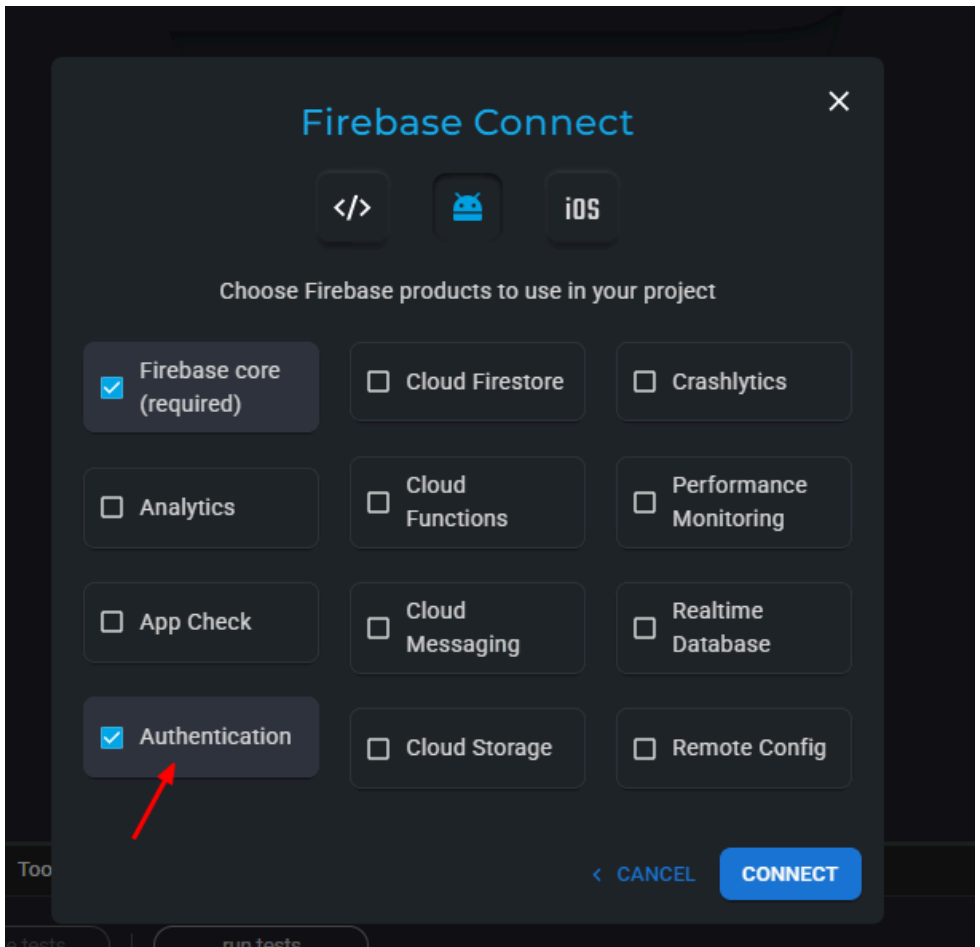
43 - Escolha Android ou IOS:



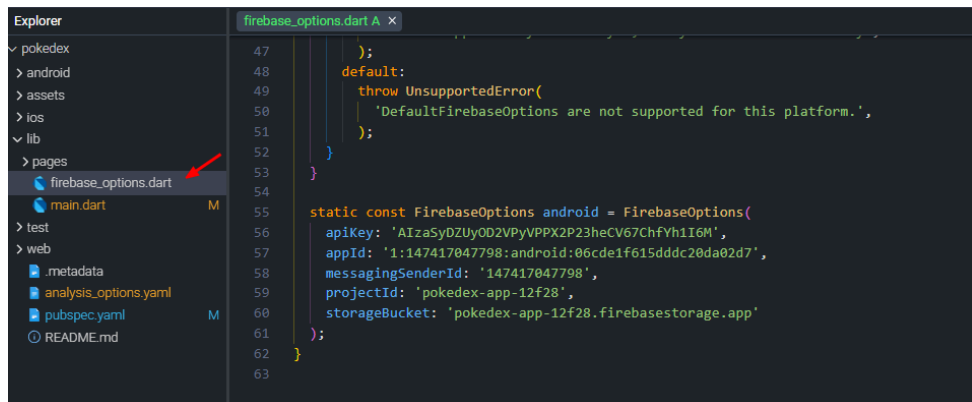
44 - Arraste o arquivo google-services.json que geramos nos passos anteriores. Clique em continue:



45 - Escolha os serviços que deseja integrar, em nosso caso selecione o serviço de Authentication e depois connect:



46 - Veja que após a conexão o projeto criou automaticamente um arquivo de configuração chamado `firebase_options.dart`. Aqui temos informações importantes, inclusive, caso queira no futuro publicar seu app na Play Store:



The screenshot shows an IDE interface. On the left, the 'Explorer' panel displays a project structure with folders like 'android', 'assets', 'ios', 'lib', 'pages', 'test', and 'web'. Under the 'lib' folder, 'firebase_options.dart' is highlighted with a red arrow. The main editor area shows the content of 'firebase_options.dart' with line numbers 47 to 63. The code defines a default FirebaseOptions and a static const FirebaseOptions for Android.

```
47     );
48     default:
49       throw UnsupportedError(
50         'DefaultFirebaseOptions are not supported for this platform.',
51       );
52   }
53 }
54
55 static const FirebaseOptions android = FirebaseOptions(
56   apiKey: 'AIzaSyDZUyOD2VPyVPPX2P23heCV67ChfYh1I6M',
57   appId: '1:147417047798:android:06cde1f615dddc20da02d7',
58   messagingSenderId: '147417047798',
59   projectId: 'pokedex-app-12f28',
60   storageBucket: 'pokedex-app-12f28.firebaseioapp.appspot.com',
61 );
62 }
63
```

47 - Abra o arquivo main.dart, adicione algumas importações do firebase (linha 2 e 3) e deixe-o como abaixo. Dentro de main estamos inicializando os serviços do Firebase (ensureInitialized) e inicializando o serviço de acordo com a plataforma (Android ou IOS), lembrando que esse código atende as duas plataformas:

```
main.dart M x
1 //Firebase
2 import 'package:firebase_auth/firebase_auth.dart';
3 import 'package:firebase_core/firebase_core.dart';
4
5 //Material UI
6 import 'package:flutter/material.dart';
7
8 //Páginas Internas
9 import 'package:pokedex/firebase_options.dart';
10 import 'package:pokedex/pages/login_page.dart';
11
12 void main() async {
13   WidgetsFlutterBinding.ensureInitialized();
14   await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
15   runApp(const MyApp());
16 }
17
18 class MyApp extends StatelessWidget {
19   const MyApp({super.key});
20
21   @override
22   Widget build(BuildContext context) {
23     return const MaterialApp(
24       debugShowCheckedModeBanner: false,
25       home: LoginPage(),
26     ); // MaterialApp
27   }
28 }
29
```